SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code: DSA03 Course Title: Natural Language Processing

CO Assessed: CO1 – Imitation (L1) Assessment: Assessment Tool 3 – Coding Challenge

Weightage: 30% of CIA

CO5 Statement: Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)

Student Name: Prajiith K Reg. No.: 192324013

Section / Batch: C Date: 16/10/2026

Code of Conduct

I Prajiith K 192324013 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Prajiith

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Answer 1

```
"""
Section 1: Intelligent Email and Password Validator using Regular Expressions
Validates Email, Password and Mobile Number according to the given policies.
"""
import re
import sys

# ---------------------------------------------------------------- patterns
# Email: starts with an alphabet, letters/digits/'.'/'_' before '@',
# domain contains only alphabets, extension in (com, org, edu, net, in)
EMAIL_PATTERN = re.compile(r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$")

# Password: min 8 chars, upper, lower, digit and special character (@#$%&!)
PASSWORD_PATTERN = re.compile(r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$")

# Mobile: exactly 10 digits, first digit between 6 and 9
MOBILE_PATTERN = re.compile(r"^[6-9]\d{9}$")


# ---------------------------------------------------------------- validators
def validate_email(email: str) -> str:
    return "Valid Email" if EMAIL_PATTERN.match(email) else "Invalid Email"


def validate_password(password: str) -> str:
    return "Strong Password" if PASSWORD_PATTERN.match(password) else "Weak Password"


def validate_mobile(mobile: str) -> str:
    return "Valid Mobile Number" if MOBILE_PATTERN.match(mobile) else "Invalid Mobile Number"

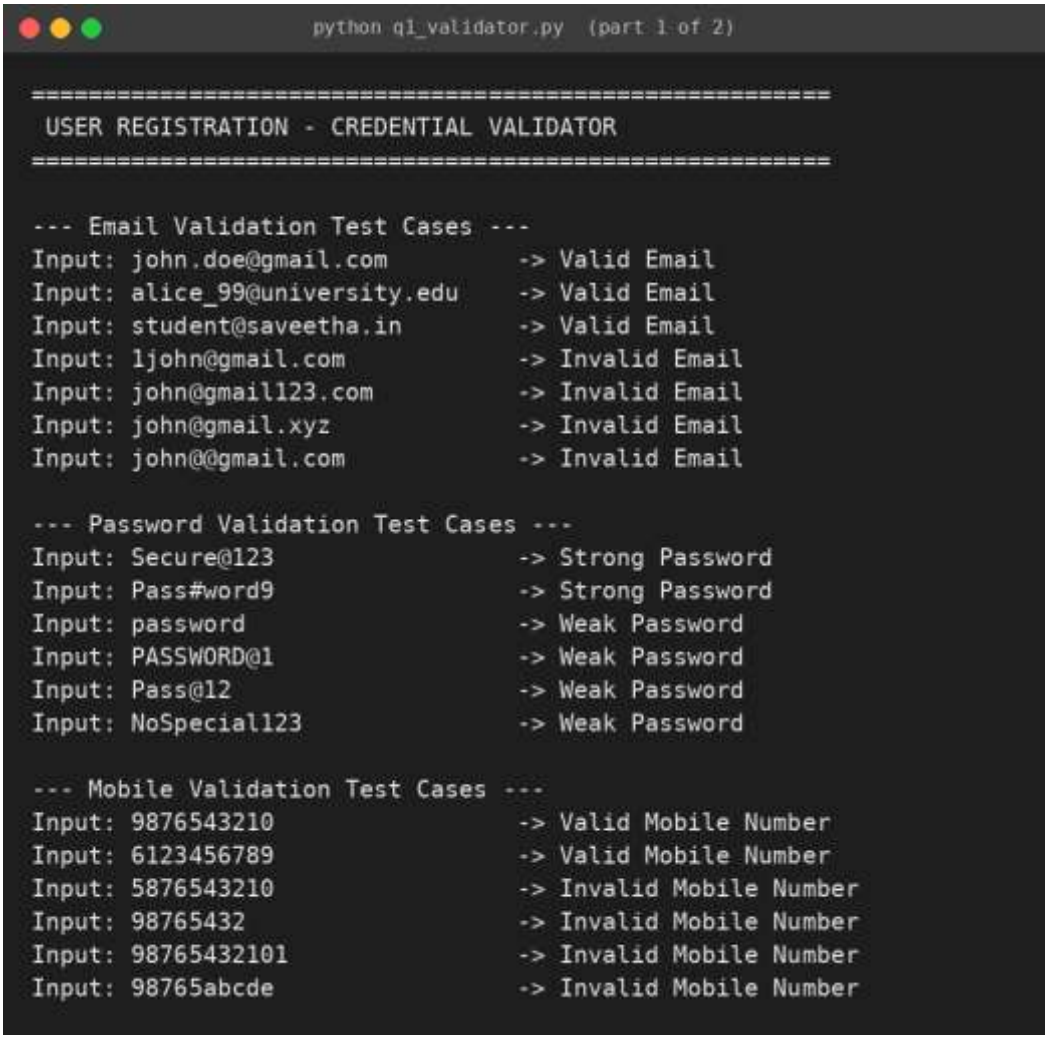

def run_test_cases() -> None:
    """Display input, processing (pattern used) and output for test cases."""
    tests = [
        ("Email", validate_email, [
            "john.doe@gmail.com", "alice_99@university.edu", "student@saveetha.in",
            "1john@gmail.com", "john@gmail123.com", "john@gmail.xyz", "john@@gmail.com"]),
        ("Password", validate_password, [
            "Secure@123", "Pass#word9", "password", "PASSWORD@1", "Pass@12", "NoSpecial123"]),
        ("Mobile", validate_mobile, [
            "9876543210", "6123456789", "5876543210", "98765432", "98765432101", "98765abcde"]),
    ]
    for name, func, cases in tests:
        print(f"--- {name} Validation Test Cases ---")
        for value in cases:
            print(f"Input: {value:<26} -> {func(value)}")
        print()

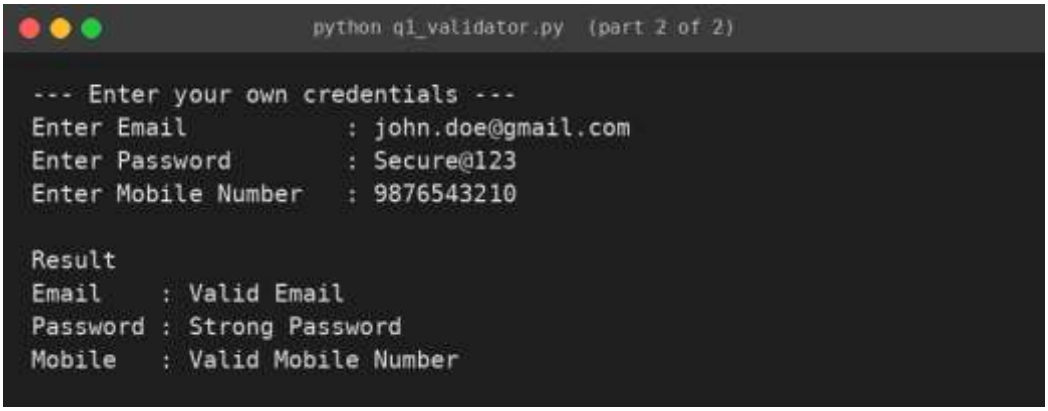

def read_input(prompt: str) -> str:
    """Read a line; echo it when input is piped so the run is self-explanatory."""
    try:
        value = input(prompt)
    except EOFError:
        value = ""
    if not sys.stdin.isatty():
```

```
        print(value)
    return value.strip()


def main() -> None:
    print("=" * 56)
    print(" USER REGISTRATION - CREDENTIAL VALIDATOR")
    print("=" * 56)
    print()
    run_test_cases()
    print("--- Enter your own credentials ---")
    email = read_input("Enter Email           : ")
    password = read_input("Enter Password        : ")
    mobile = read_input("Enter Mobile Number   : ")
    print()
    print("Result")
    print("Email    :", validate_email(email))
    print("Password :", validate_password(password))
    print("Mobile   :", validate_mobile(mobile))


if __name__ == "__main__":
    main()
```





Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

Accept the DFA description

States

Input alphabet

Transition table

Initial state

Final state(s)

Accept multiple input strings.

Simulate state transitions.

Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

q0 → q1 → q0 → q1 → q2

Accepted

Answer 2

```
"""
Section 2: Deterministic Finite Automata (DFA) Simulator
Accepts a DFA description (states, alphabet, transition table, initial state,
final states) and simulates any number of input strings.
"""
import sys


class DFA:
    def __init__(self, states, alphabet, transitions, start, finals):
        self.states = states
        self.alphabet = alphabet
        self.transitions = transitions      # {(state, symbol): next_state}
        self.start = start
        self.finals = finals

    def simulate(self, text: str):
        """Return (path, accepted, message)."""
        state = self.start
        path = [state]
```

```
        for symbol in text:
            if symbol not in self.alphabet:
                return path, False, f"Symbol '{symbol}' is not in the input alphabet"
            if (state, symbol) not in self.transitions:
                return path, False, f"No transition from {state} on '{symbol}'"
            state = self.transitions[(state, symbol)]
            path.append(state)
        return path, state in self.finals, ""

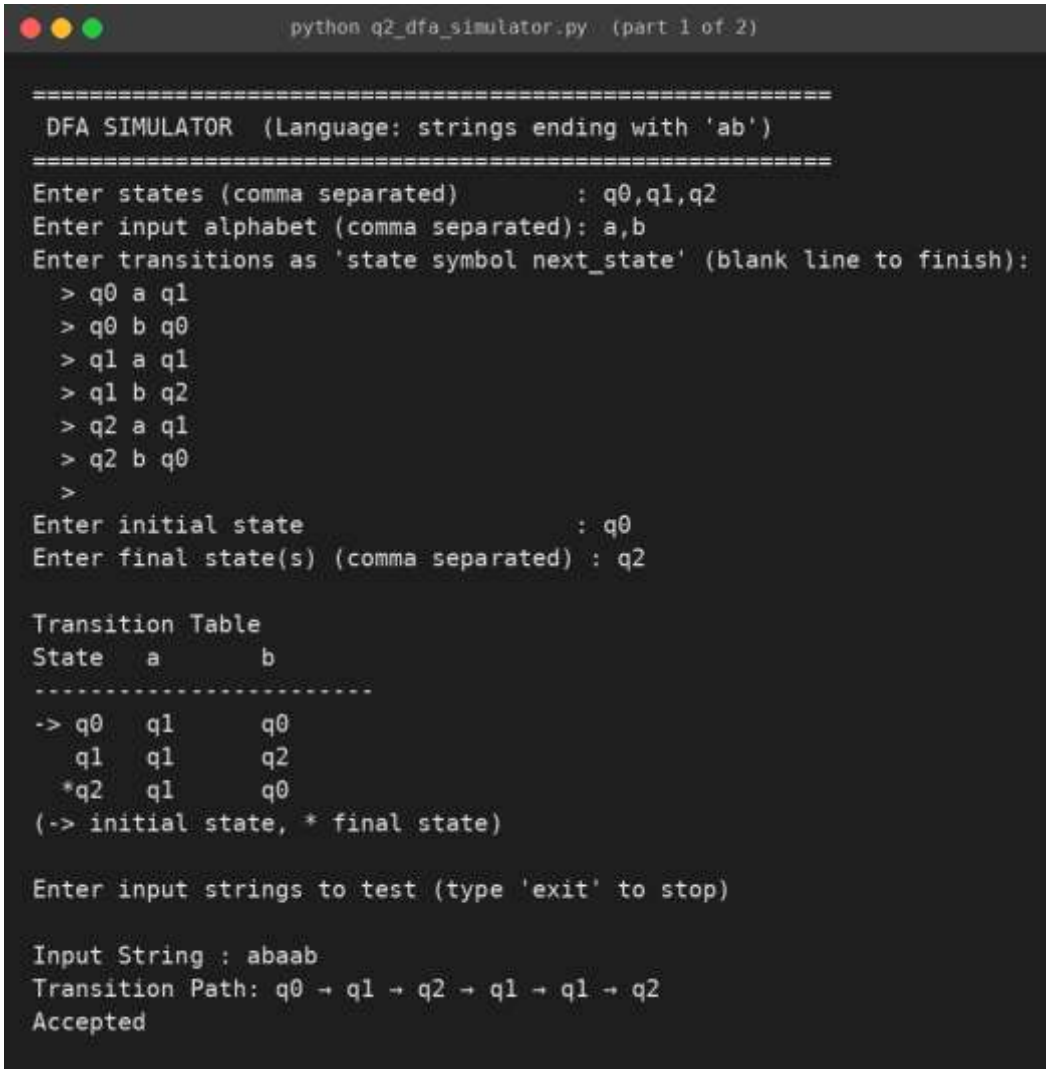

def read_input(prompt: str) -> str:
    try:
        value = input(prompt)
    except EOFError:
        value = "exit"
    if not sys.stdin.isatty():
        print(value)
    return value.strip()

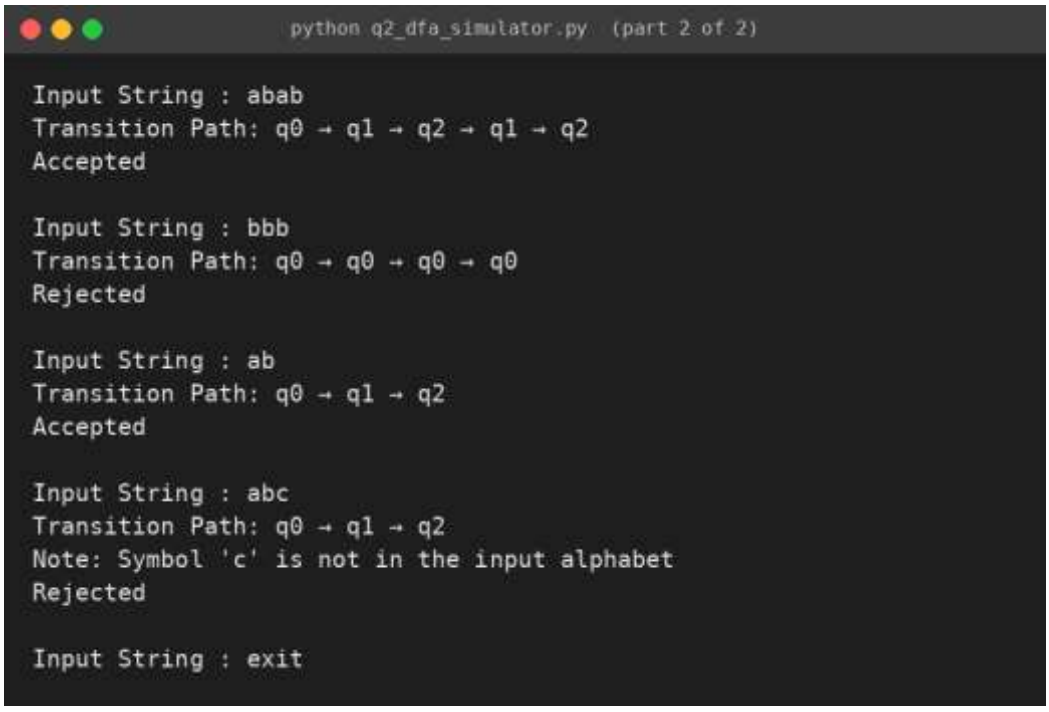

def read_dfa() -> DFA:
    """Read the DFA description from the user with validation."""
    states = [s.strip() for s in read_input("Enter states (comma separated)        : ").split(",") if s.strip()]
    alphabet = [a.strip() for a in read_input("Enter input alphabet (comma separated): ").split(",") if a.strip()]
    print("Enter transitions as 'state symbol next_state' (blank line to finish):")
    transitions = {}
    while True:
        line = read_input("  > ")
        if not line:
            break
        parts = line.split()
        if len(parts) != 3:
            print("    Invalid format, use: state symbol next_state")
            continue
        src, sym, dst = parts
        if src not in states or dst not in states or sym not in alphabet:
            print("    Unknown state or symbol, transition ignored")
            continue
        transitions[(src, sym)] = dst
    start = read_input("Enter initial state                  : ")
    finals = [f.strip() for f in read_input("Enter final state(s) (comma separated) : ").split(",") if f.strip()]
    if start not in states or not set(finals) <= set(states):
        raise ValueError("Initial/final state is not in the set of states")
    return DFA(states, alphabet, transitions, start, finals)


def print_table(dfa: DFA) -> None:
    print("\nTransition Table")
    header = "State".ljust(8) + "".join(sym.ljust(8) for sym in dfa.alphabet)
    print(header)
    print("-" * len(header))
    for s in dfa.states:
        mark = ("->" if s == dfa.start else "  ") + ("*" if s in dfa.finals else " ")
        row = (mark + s).ljust(8) + "".join(dfa.transitions.get((s, a), "-").ljust(8) for a in dfa.alphabet)
        print(row)
    print("(-> initial state, * final state)\n")


def main() -> None:
    print("=" * 56)
    print(" DFA SIMULATOR  (Language: strings ending with 'ab')")
    print("=" * 56)
    try:
        dfa = read_dfa()
    except ValueError as err:
        print("Error:", err)
        return
    print_table(dfa)
    print("Enter input strings to test (type 'exit' to stop)")
    while True:
        text = read_input("\nInput String : ")
        if text.lower() == "exit":
            break
        path, accepted, message = dfa.simulate(text)
        print("Transition Path:", " \u2192 ".join(path))
        if message:
            print("Note:", message)
        print("Accepted" if accepted else "Rejected")


if __name__ == "__main__":
    main()
```





Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.
The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input

Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu

1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Answer 3

```
"""
Section 3: Smart Pattern Matching Engine using Regular Expressions
Supports Word, Prefix, Suffix, Date, Phone Number, Hashtag and Mention search.
"""
import re
import sys

SAMPLE_TEXT = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""

DATE_PATTERN = r"\b\d{1,2}/\d{1,2}/\d{4}\b"
PHONE_PATTERN = r"\b[6-9]\d{9}\b"
HASHTAG_PATTERN = r"#\w+"
MENTION_PATTERN = r"@\w+"


def search_pattern(pattern: str, text: str):
    return re.findall(pattern, text, flags=re.IGNORECASE)


def search_word(word: str, text: str):
    return re.findall(rf"\b{re.escape(word)}\b", text, flags=re.IGNORECASE)


def search_prefix(prefix: str, text: str):
    return re.findall(rf"\b{re.escape(prefix)}\w*", text, flags=re.IGNORECASE)


def search_suffix(suffix: str, text: str):
    return re.findall(rf"\b\w*{re.escape(suffix)}\b", text, flags=re.IGNORECASE)


def read_input(prompt: str) -> str:
    try:
        value = input(prompt)
    except EOFError:
```

```
        value = "0"
    if not sys.stdin.isatty():
        print(value)
    return value.strip()


def show(title: str, matches) -> None:
    if matches:
        print(f"{title} found ({len(matches)}):")
        for item in matches:
            print("   ", item)
    else:
        print(f"No {title.lower()} found")


def menu() -> None:
    print("\nUser Menu")
    print("1.Search Date")
    print("2.Search Phone Number")
    print("3.Search Hashtag")
    print("4.Search Mention")
    print("5.Search Prefix")
    print("6.Search Suffix")
    print("7.Search Word")
    print("0.Exit")


def main() -> None:
    print("=" * 56)
    print(" SMART PATTERN MATCHING ENGINE")
    print("=" * 56)
    print("Input Text:")
    print(SAMPLE_TEXT)
    text = SAMPLE_TEXT
    while True:
        menu()
        choice = read_input("Enter your choice: ")
        if choice == "1":
            show("Date", search_pattern(DATE_PATTERN, text))
        elif choice == "2":
            show("Phone Number", search_pattern(PHONE_PATTERN, text))
        elif choice == "3":
            show("Hashtag", search_pattern(HASHTAG_PATTERN, text))
        elif choice == "4":
            show("Mention", search_pattern(MENTION_PATTERN, text))
        elif choice == "5":
            prefix = read_input("Enter prefix: ")
            show("Prefix match", search_prefix(prefix, text)) if prefix else print("Prefix cannot be empty")
        elif choice == "6":
            suffix = read_input("Enter suffix: ")
            show("Suffix match", search_suffix(suffix, text)) if suffix else print("Suffix cannot be empty")
        elif choice == "7":
            word = read_input("Enter word: ")
            show("Word match", search_word(word, text)) if word else print("Word cannot be empty")
        elif choice == "0":
            print("Exiting...")
            break
        else:
            print("Invalid choice, please select 0-7")


if __name__ == "__main__":
    main()
```

python q3_pattern_search.py (part 1 of 5)

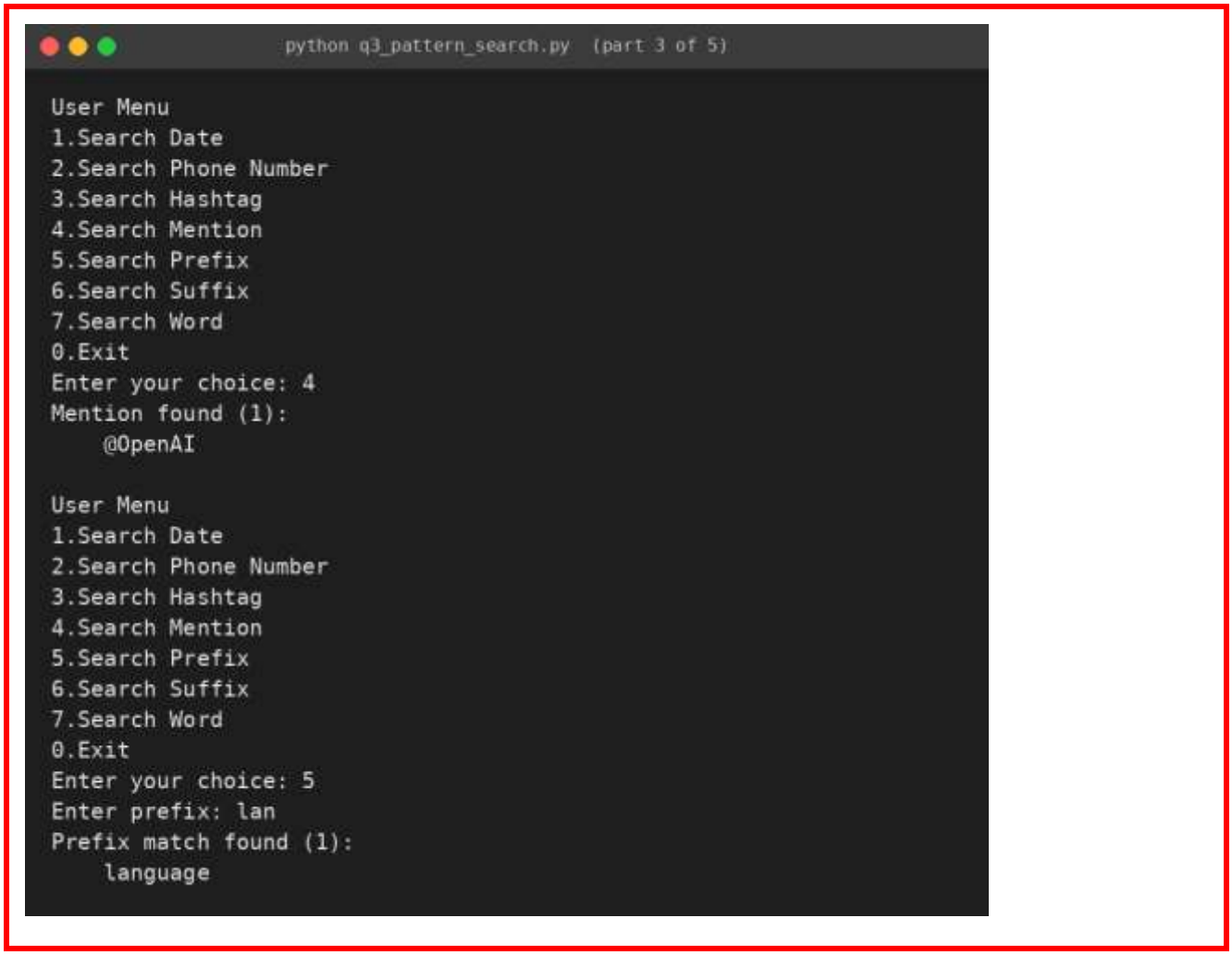
```
============================================================
 SMART PATTERN MATCHING ENGINE
============================================================
Input Text:
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 1
Date found (1):
    12/09/2026
```

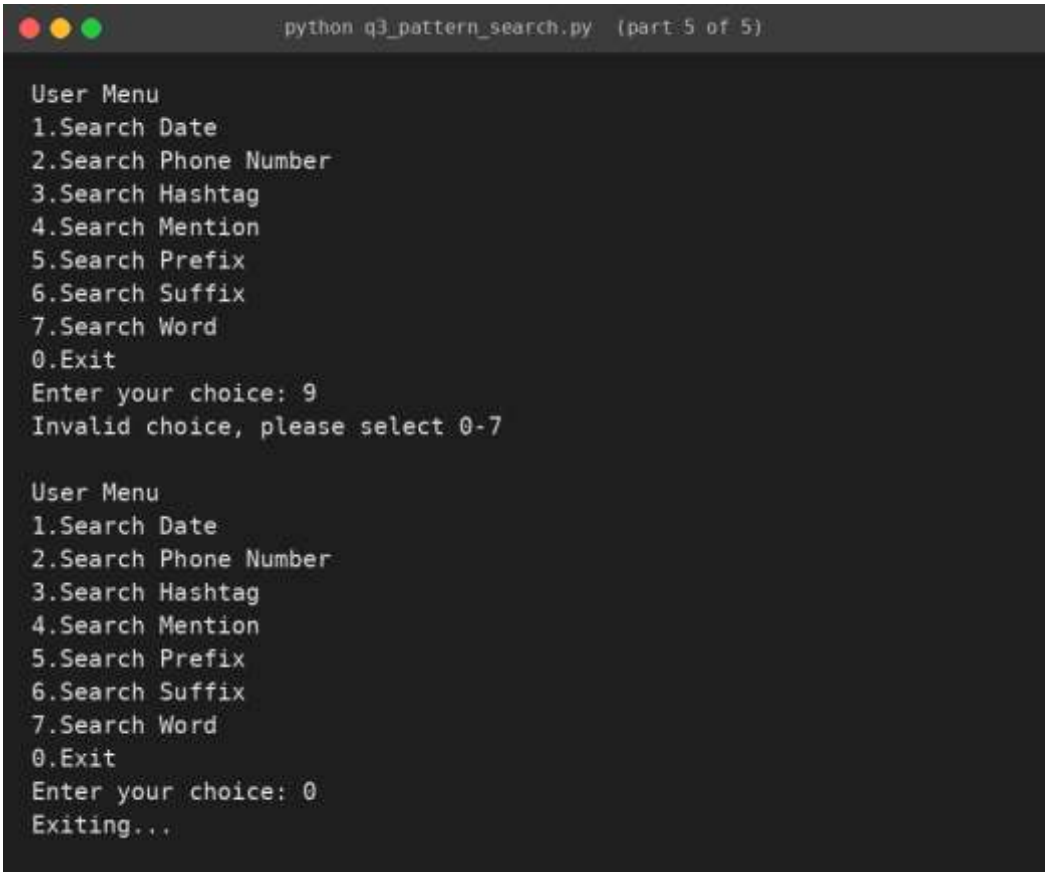
python q3_pattern_search.py (part 2 of 5)

```
User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 2
Phone Number found (1):
    9876543210

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 3
Hashtag found (1):
    #NLP
```


python q3_pattern_search.py (part 3 of 5)
User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 4
Mention found (1):
@OpenAI
User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 5
Enter prefix: lan
Prefix match found (1):
language


python q3_pattern_search.py (part 4 of 5)
User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 6
Enter suffix: ing
Suffix match found (2):
Meeting
processing
User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 7
Enter word: language
Word match found (1):
language


python q3_pattern_search.py (part 5 of 5)
User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 9
Invalid choice, please select 0-7
User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit
Enter your choice: 0
Exiting...

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge
Signature: _____
Date: _____

Course Coordinator
Signature: _____
Date: _____

Program Director
Signature: _____
Date: _____